School of Computer Science
Faculty of Science and Engineeing
University Of Nottingham
Malaysia

UG FINAL YEAR DISSERTATION REPORT

Hybrid YOLO–UNet++ Framework for Efficient Brain Tumor Localization and Segmentation in MRI Scans

Student's name : Mohammed Samir Ahmed Dasouqi

Student Number : 20409843

Supervisor Name : Dr IMAN LIAO

Year : 2025

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF BACHELOR OF SCIENCE IN SOFTWARE ENGINEERING (HONS)
THE UNIVERSITY OF NOTTINGHAM

Hybrid YOLO–UNet++ Framework for Efficient Brain Tumor Localization and Segmentation in MRI Scans

Submitted in May 2025, in partial fulfillment of the conditions of the award of the degrees B.Sc.

Mohammed Samir Ahmed Dasouqi
School of Computer Science
Faculty of Science and Engineering
University of Nottingham
Malaysia

I hereby declare that this dissertation is all my own work, except as indicated in the text:

Signature _____

Date 04 / 05 / 2025

Abstract

This project presents a two-stage deep learning pipeline for the detection and segmentation of brain tumors in MRI scans. The proposed system integrates a RepVGG-GELAN-based YOLO model for efficient tumor localization with a custom EfficientUNet++ architecture for high-resolution semantic segmentation. The pipeline is designed to be modular, allowing each stage to be independently optimized and evaluated. The BR35H dataset was used for both training and validation, with annotation refinement applied to improve segmentation quality by excluding circular and elliptical ground truth masks. The detection model demonstrated strong performance in identifying tumor regions, while the segmentation model achieved a Dice score of 0.9375, indicating high spatial accuracy. Qualitative analysis revealed that the model could outperform certain ground truth annotations, specifically in cases with imprecise or overly generalized labels. The results validate the effectiveness of combining advanced YOLO and UNet variants in a clinical imaging context and highlight the potential of lightweight, accurate pipelines in supporting medical diagnosis.

Table of Contents

List Of Figures

List Of Tables

1. Introduction

Fast growth of artificial intelligence and machine learning in recent years led to advancements in the medical imaging field by introducing faster and more efficient AI models for tumor detection and segmentation. While manual analysis by radiologists is always necessary, AI models can offer quick analysis of medical images, assisting radiologists find abnormalities quicker and possibly detect ones that may be oversighted due to human error. By processing images in seconds rather than minutes and possibly hours, AI-powered systems can support medical workers in managing high workloads and reducing diagnostic delays.

In medical image analysis, segmentation networks such as U-Net have long been the standard approach for pixel-wise labeling. Introduced in 2015, U-Net was specifically designed for biomedical image segmentation and has since become widely adopted due to its encoder-decoder structure and ability to produce finely detailed masks from limited training data. Variants such as U-Net++ and architectures integrating advanced encoders like EfficientNet have further improved segmentation accuracy in challenging clinical scenarios.

More recently, object detection models like the "You Only Look Once" (YOLO) series have gained attention in the medical domain due to their high-speed inference and real-time detection capabilities. While YOLO was originally developed for general-purpose object detection, its ability to quickly locate regions of interest has made it compelling for medical tasks such as tumor localization. However, YOLO's bounding box predictions lack the granularity needed for detailed tumor segmentation, making it more suitable as a preliminary detection step rather than a standalone solution in clinical workflows.

In this project, we aim to combine the strengths of both approaches by integrating a YOLO-based detection mechanism with a U-Net segmentation model. The proposed method first employs YOLO to localize potential tumor regions via bounding boxes, which are then cropped and padded then passed to a U-Net-based architecture for precise segmentation. This hybrid approach seeks to improve overall accuracy while maintaining computational efficiency.

The project builds upon Balakrishnan's (2024) RepVGG-GELAN architecture, which is a high-performance YOLO variant serving as the detection backbone. By leveraging this dual-stage system, the project aspires to bridge the gap between coarse detection and fine-grained segmentation, ultimately contributing to more effective AI-assisted medical diagnosis.

1.2 Motivation

Brain tumors are among the most serious and life-threatening medical conditions, requiring timely and accurate diagnosis for effective treatment planning. MRI imaging plays a critical role in detecting and analyzing brain tumors, but manual interpretation of MRI scans is time-consuming, prone to variability, and demands significant expertise from radiologists. Additionally, tumors can vary greatly in size, shape, and location, making consistent detection and segmentation a complex task.

Automating this process using deep learning offers a promising solution, yet many existing models either focus solely on segmentation or rely on complex end-to-end designs that

are difficult to interpret or modify. Additionally, the presence of noisy or inconsistent annotations in public datasets introduces further challenges for model training and evaluation.

This project addresses these issues by developing a modular two-stage pipeline that first localizes tumors using an object detection model (RepVGG-GELAN) and then applies a refined segmentation model (EfficientUNet++) to generate accurate tumor masks. The approach balances performance and flexibility, enabling better, visualization, and model customization. By aiming for both accuracy and efficiency, this system contributes toward improving the reliability and accessibility of automated brain tumor analysis, particularly in clinical or resource-limited environments.

2. Literature Review

2.1 YOLO for Object Detection

Object detection is a fundamental task in computer vision, where the goal is to identify and localize an object, or multiple, within an image. A major breakthrough in this field came with the introduction of the YOLO (You Only Look Once) framework by Redmon et al. (2016). Unlike traditional multi-stage detectors that perform region proposal and classification separately, YOLO reformulated object detection as a single-stage regression problem, predicting bounding boxes and class probabilities directly from full images in one pass.

The YOLO architecture divides an input image into a grid and predicts bounding boxes and class scores for each cell simultaneously. This design choice allows the model to perform detection in real time, offering a strong balance between speed and accuracy. Over the years, the YOLO family has evolved significantly from YOLOv1 through YOLOv9, introducing architectural enhancements such as anchor boxes, residual connections, decoupled heads, and dynamic label assignment to improve precision, robustness, and training stability.

The high inference speed of YOLO makes it particularly attractive in medical applications where rapid decision-making is crucial, such as tumor localization in radiological workflows. By applying YOLO to detect the location of brain tumors in MRI images, clinicians and downstream systems can focus only on regions of interest, enabling faster and more focused segmentation and diagnosis.

In this project, the detection component is based on RepVGG-GELAN, a recent YOLO-style model introduced by Balakrishnan and Sengar (2024). This architecture combines the reparameterization benefits of RepVGG with the efficient feature aggregation of GELAN, yielding a fast and accurate detector well-suited for real-time medical image processing. The model was adapted to not only generate bounding boxes but also to extract and export padded tumor regions, which are then passed to a segmentation model for further analysis.

2.2 Introduction to U-Net

The rapid growth of artificial intelligence and machine learning has transformed the field of medical image analysis, more specifically in the area of image segmentation. A breakthrough came when Ronneberger et al. (2015) introduced the U-Net architecture. The U-Net architecture was designed specifically for biomedical image segmentation tasks. U-Net

established itself as a landmark model by addressing the challenge of performing pixel-wise classification from a limited number of annotated medical images.

The U-Net architecture follows a symmetric encoder-decoder design. The encoder path progressively reduces spatial dimensions through convolutional and pooling layers, allowing the network to capture increasingly abstract and semantic feature representations. In contrast, the decoder path performs upsampling operations to gradually reconstruct the segmentation map and restore spatial resolution. However, a direct upsampling of deeply abstracted features often leads to a loss of fine-grained spatial details, which are critical for precise boundary delineation in medical images.

To overcome this challenge, U-Net introduced skip connections between the encoder and decoder at corresponding resolution levels. These connections pass feature maps from earlier layers of the encoder directly into later layers of the decoder. By concatenating high-resolution features from the encoder with upsampled features in the decoder, the model preserves fine spatial information that would otherwise be lost during downsampling. This innovation was crucial for enabling U-Net to achieve highly accurate segmentations even on small or thin structures, such as blood vessels, lesions, or tumor margins, where pixel-level precision is necessary.

The characteristic U-shaped structure of the architecture, together with the skip connections linking encoder and decoder paths, is illustrated in Figure 1.

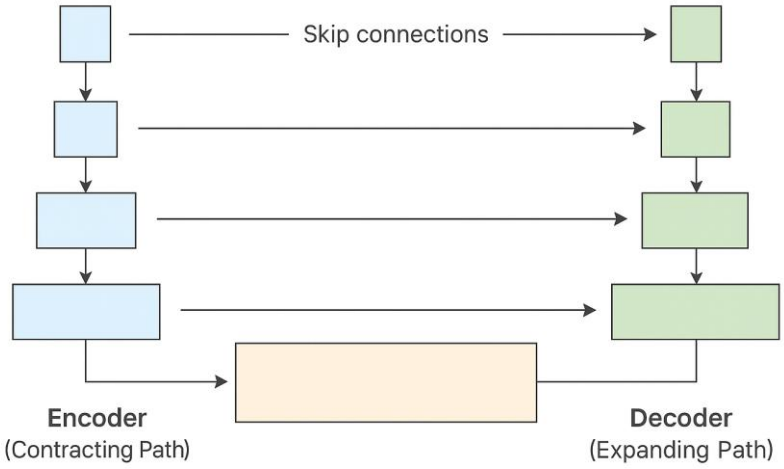

Figure 1 Simplified schematic of the U-Net architecture, showing the encoder (contracting path), decoder (expanding path), and skip connections linking corresponding layers.

Skip connections also helped solve the vanishing gradient problem, where early parts of the network struggle to learn properly, by making it easier for information to flow backward during training. Because of this, U-Net could train well even when only a small number of labeled images were available, making it a good choice for medical tasks where large datasets are often hard to get.

In addition, U-Net's design enabled effective training through the use of data augmentation techniques, including elastic deformations, random crops, and intensity variations. These augmentations helped to artificially expand limited datasets, making the model able to generalize better and avoid overfitting.

In their original experiments, Ronneberger et al. demonstrated the effectiveness of U-Net across multiple biomedical segmentation tasks, such as the segmentation of neuronal structures in electron microscopy stacks and cell tracking in light microscopy images. The model achieved substantial improvements over traditional patch-based classification methods, both in segmentation accuracy and computational efficiency.

Prior to U-Net, many segmentation pipelines relied on sliding window approaches, where each patch of the image was classified independently. Such methods were not only computationally expensive but also suffered from inconsistent predictions across overlapping patches. U-Net's end-to-end, fully convolutional design eliminated the need for sliding windows, providing a unified solution that was faster and produced more coherent and spatially consistent segmentation outputs.

By combining deep semantic understanding with precise spatial localization, U-Net set a new standard for medical image segmentation and established itself as the foundational architecture upon which numerous subsequent models and improvements have been built.

2.3 Challenges and Motivations for Improvement

Although U-Net achieved impressive results in medical image segmentation, its original design also revealed certain limitations that motivated researchers to develop improved versions. One major challenge was that the basic U-Net architecture, despite its skip connections, sometimes struggled to fully bridge the semantic gap between the encoder and decoder feature maps. Features extracted at different stages of the encoder can vary widely in their level of abstraction, and simply concatenating these features may not always be optimal for producing accurate segmentation boundaries (Zhou et al., 2018).

Another important issue was related to the network's capacity and efficiency. As medical imaging tasks became more complex, with higher-resolution inputs and a greater diversity of structures to segment, the original U-Net sometimes lacked the depth and representational power needed to handle complex cases. Additionally, although U-Net performed well with limited data, its relatively simple design made it more prone to overfitting on very small datasets without additional regularization or architectural enhancements.

These challenges sparked a wave of research aimed at improving U-Net's ability to handle more complex medical images while maintaining computational efficiency. Researchers began exploring ways to make the encoder-decoder connections more intelligent, to improve feature fusion, and to increase the network's overall capacity without significantly increasing training difficulty or computational load.

One of the most influential efforts in this direction was the development of U-Net++, which introduced the idea of nested skip connections to better align feature maps at different levels of the encoder and decoder. Later enhancements involved integrating stronger backbone networks, such as EfficientNet, to improve feature extraction and model scaling without unnecessarily increasing the model size or computational cost.

Thus, the evolution of U-Net-based architectures has been largely driven by the need to achieve better segmentation accuracy, more robust feature fusion, and greater efficiency, particularly in challenging biomedical imaging scenarios.

2.4 U-Net++

To address the challenges observed in the original U-Net design, Zhou et al. proposed U-Net++, a significant architectural improvement aimed at enhancing feature fusion and segmentation performance [Zhou et al., 2018]. The core idea behind U-Net++ was to introduce nested and dense skip connections that more effectively bridge the semantic gap between the encoder and decoder feature maps.

In the standard U-Net, skip connections directly link encoder features to corresponding decoder layers. However, the feature maps at different stages of the encoder and decoder are often semantically inconsistent: encoder features are abstract and high-level, while decoder features require more spatial and localized information. Simply concatenating them, as done in vanilla U-Net, can confuse the decoder and lead to blurred boundaries and reduced segmentation accuracy. U-Net++ solves this by adding a series of intermediate convolutional blocks that progressively refine the encoder features before passing them to the decoder. This gradual refinement reduces the semantic gap and results in better alignment between encoder and decoder information.

The structural differences between U-Net and U-Net++ are illustrated in Figure 2, highlighting the nested skip connections and dense fusion paths that distinguish the improved design.

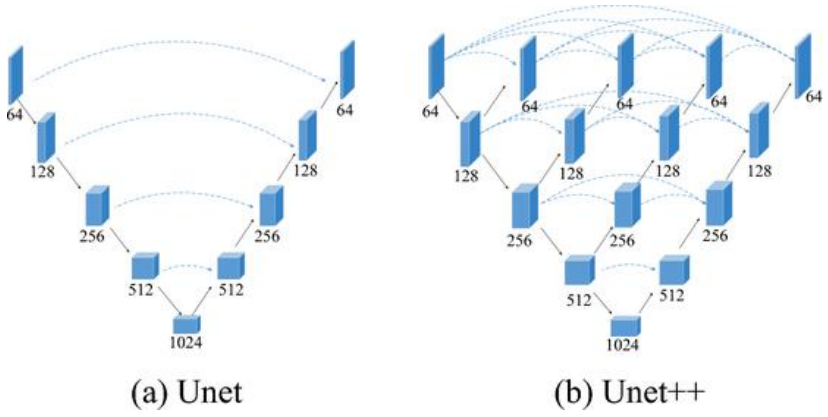

Figure 2: Comparison of U-Net and U-Net++ architectures. U-Net uses direct skip connections between encoder and decoder layers, while U-Net++ introduces nested and dense skip pathways to enhance feature fusion and reduce the semantic gap. (Source: Liu et al., 2024)

Architecturally, U-Net++ resembles a nested U structure, where each skip pathway is composed of multiple convolutional units, creating deeper feature fusion along each skip route. This dense connection design enables more effective multi-scale aggregation and leads to more accurate segmentation outputs, especially for complex medical images with small or irregular structures.

Another key innovation introduced in U-Net++ is deep supervision. Instead of supervising the model only at the final output layer, U-Net++ attaches auxiliary outputs to intermediate decoder nodes at different depths. During training, these intermediate outputs are supervised by the ground truth masks, helping guide the network to learn more discriminative features early on. Deep supervision improves model convergence, reduces overfitting on small datasets, and makes the network more robust during inference.

Additionally, U-Net++ was designed to allow pruning flexibility. Because the network is nested, sub-networks of varying depths can be extracted based on the available data size or computational constraints. For small datasets or resource-constrained environments, a shallower version of U-Net++ can be trained, while for larger, complex datasets, the full nested model can be utilized.

In practical benchmarks, U-Net++ demonstrated superior performance compared to the original U-Net, particularly on tasks involving highly detailed structures or complex boundaries, such as lung nodule segmentation and colonoscopy lesion detection. These results showed that smarter skip connections, deeper feature alignment, and multi-scale learning could significantly boost the effectiveness of medical image segmentation models.

The success of U-Net++ paved way for further improvements, including the integration of more powerful encoders like EfficientNet to enhance feature extraction even further.

2.5 Integration of Efficient Encoders

As segmentation tasks grew increasingly complex and demanded stronger feature extraction capabilities, researchers began exploring ways to replace U-Net's relatively simple encoder with more powerful and efficient backbone networks. One significant advancement was the introduction of EfficientNet by Tan and Le (2019), a family of convolutional neural networks that achieved superior accuracy and efficiency by using a principled compound scaling method.

EfficientNet scales network depth, width, and resolution systematically, rather than arbitrarily, allowing it to achieve better performance with fewer parameters compared to traditional architectures like ResNet or VGG. Among the EfficientNet models, EfficientNet-B0 stands out as a lightweight yet powerful option, balancing computational efficiency and representational strength, making it highly suitable for medical imaging tasks where resources and inference time are important considerations.

Incorporating EfficientNet as the encoder within U-Net-based architectures became a natural next step to enhance segmentation models. The resulting models, often referred to as EfficientUNet or EfficientUNet++ when combined with nested skip designs, replaced U-Net's original convolutional encoder with an EfficientNet backbone. This integration brought substantial benefits:

- Improved feature extraction across scales.
- Better representation of small or complex structures.
- Reduced model size and faster convergence during training.

Specifically, the EfficientUNet++ architecture used in this project leverages EfficientNet-B0 as its encoder, combined with the dense skip connections and deep supervision principles of U-Net++. This combination enhances the network's ability to capture detailed contextual information while remaining computationally efficient, making it particularly suitable for tasks like brain tumor segmentation, where precision and speed are both critical.

Thus, by integrating stronger backbone encoders like EfficientNet-B0 into the already improved U-Net++ structure, modern segmentation models have achieved state-of-the-art performance in a variety of medical imaging challenges.

2.6 EfficientUNet++

EfficientUNet++ represents a merging of multiple architectural improvements aimed at achieving high segmentation accuracy while maintaining computational efficiency. It combines the strengths of EfficientNet-B0 as a feature-rich encoder and the nested skip connection design of U-Net++, forming a powerful framework particularly well-suited for medical image segmentation tasks such as brain tumor detection.

In this architecture that we implemented, EfficientNet-B0 serves as the encoder, extracting multi-scale feature maps from the input image. These features are passed through a series of nested decoder blocks that mirror the structure of U-Net++, enabling dense feature fusion across different layers. Each decoder stage integrates upsampled features with corresponding encoder features, progressively reconstructing the segmentation map with enhanced spatial precision.

What distinguishes EfficientUNet++ from its predecessors is its ability to combine deep context understanding with fine boundary localization while remaining lightweight. The use of pretrained EfficientNet weights provides a strong initial representation, which is particularly beneficial in medical settings where annotated data may be limited. Meanwhile, the nested decoder structure allows for better gradient flow and more accurate learning of complex anatomical structures.

EfficientUNet++ has shown strong performance across a variety of segmentation benchmarks, including organ delineation, lesion detection, and tumor segmentation. Its modular design also makes it adaptable to different image sizes and complexity levels, allowing researchers to scale the model as needed.

In this project, the EfficientUNet++ implementation was tailored to accept padded tumor region crops generated by the preceding YOLO-based detection model. The segmentation network was trained to predict a binary tumor mask within each cropped region, leveraging the model's ability to capture both global context and local detail. This design ensures that segmentation is focused and precise, especially in cases involving small or irregular tumors.

With this theoretical foundation established, we now examine recent studies and implementations that combine YOLO and U-Net in medical image analysis.

3. Related Work

This section highlights prior work in tumor detection and segmentation, particularly approaches that use either YOLO, U-Net, or a hybrid of both.

3.1 Medical Image Segmentation with U-Net and Its Variants

Medical image segmentation is crucial in diagnosing and treating diseases, particularly in oncology, where precise delineation of tumors is critical. Introduced by Ronneberger et al. (2015), U-Net revolutionized biomedical segmentation tasks by effectively addressing the problem of limited annotated training data. Its symmetrical encoder-decoder architecture, featuring skip connections, captures both high-level contextual information and detailed spatial resolution, resulting in highly accurate segmentation masks.

Since its inception, several variants of U-Net have emerged, enhancing its accuracy and robustness. U-Net++ introduced nested dense connections and deep supervision, improving segmentation performance in complex medical scenarios (Zhou et al., 2018). Similarly, architectures like EfficientNet have been integrated as encoders within U-Net-based models, significantly enhancing computational efficiency and feature extraction capabilities.

3.2 Real-time Detection with YOLO Architectures

In recent years, real-time object detection algorithms, especially from the YOLO ("You Only Look Once") family, have attracted considerable attention due to their ability to quickly detect objects within an image. Initially proposed by Redmon et al. (2016), YOLO introduced a single-step detection framework that significantly increased inference speed while maintaining competitive accuracy compared to previous multi-stage detectors.

Following the initial YOLO architecture, numerous improvements have been proposed. YOLOv4 introduced advancements such as Cross Stage Partial connections (CSPNet) and spatial pyramid pooling (SPP) to further enhance accuracy without compromising speed. Later, YOLOv7 incorporated a trainable "bag-of-freebies" approach, achieving state-of-the-art performance in real-time object detection. YOLOv8 further improved upon these advancements by adopting an anchor-free detection mechanism, simplifying the model's architecture, and reducing computational overhead. The recent YOLOv9 introduced the Generalized Efficient Layer Aggregation Network (GELAN), which improves backbone efficiency and detection performance through enhanced feature aggregation and streamlined convolutional paths.

3.3 Hybrid Approaches Combining Detection and Segmentation

While YOLO-based models offer quick and high localization performance, they lack the pixel-level precision required for detailed tumor segmentation. To overcome this limitation, hybrid approaches have been proposed that combine object detection models like YOLO for localization with segmentation models like U-Net for fine-grained mask generation.

A notable example is the work by Iriawan et al. (2024), where a YOLO-UNet hybrid architecture was developed specifically for MRI brain tumor analysis. In their approach, YOLOv4

was used to first identify the approximate tumor location, which was then passed to a U-Net model for accurate segmentation. This dual-stage architecture demonstrated significant improvements in segmentation accuracy and computational efficiency, emphasizing the complementary strengths of both models. Their findings validated the practicality of this strategy for clinical applications requiring rapid and precise analysis of brain scans.

Similarly, RepVGG-GELAN represents a cutting-edge integration of efficient detection backbones with enhanced convolutional modules. This architecture combines the reparametrized convolutions of RepVGG with GELAN's layer aggregation strategy, resulting in a model that is both computationally lightweight and highly accurate. It demonstrated improvements in precision and AP50 compared to previous YOLO-based architectures, highlighting its potential for real-time medical imaging applications (Balakrishnan, 2024).

These hybrid frameworks reflect a growing consensus in the field: object detection and segmentation, when decoupled and optimized individually, can be more effective than relying on a single architecture. By allowing detection models to rapidly localize regions of interest and segmentation models to handle boundary precision, this dual-pipeline approach offers a practical balance between speed and accuracy, especially in complex tasks such as detecting small or nested tumors.

4. Methodology

The project implements a two-model machine learning pipeline for detection and segmentation of brain tumors from MRI scans. The framework combines Balakrishnan's (2024) YOLO-based object detection model RepVGG-GELAN for tumor localization with EfficientUNet++ for segmentation. The system is designed to balance computational efficiency with high spatial accuracy.

4.1 Theoretical Framework

The approach adopted in this project is based on the principle of decoupling localization and segmentation, a design strategy where object detection and pixel-level classification are handled by specialized models. Object detection algorithms such as YOLO are optimized for high-speed and coarse localization, enabling the identification of regions of interest in real time. However, they lack the spatial granularity required for detailed medical segmentation. In contrast, encoder-decoder networks such as U-Net and its variants excel at capturing multi-scale contextual features and producing precise segmentation masks, but typically operate more slowly and are sensitive to input size.

By combining the strengths of both architectures, this two-stage pipeline aims to strike a balance between computational efficiency and spatial accuracy. The detection model (YOLO-based RepVGG-GELAN) is responsible for identifying and extracting tumor regions from full MRI images, while the segmentation model (EfficientUNet++) refines this output by predicting accurate tumor boundaries at the pixel level. This modular design allows each model to operate within its optimal domain and enables faster inference, reduced false positives, and improved segmentation quality.

The overall pipeline architecture, combining object detection and segmentation, is illustrated in Figure 2.

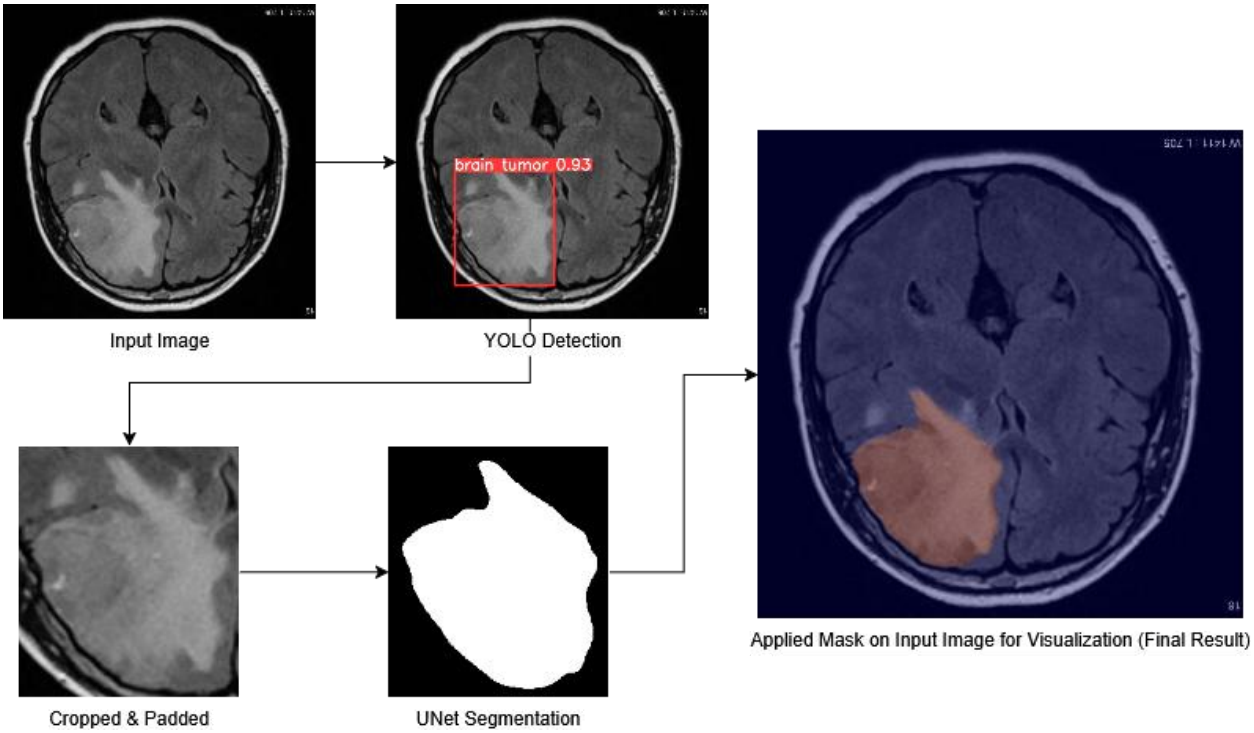

Figure 3: Schematic overview of the two-stage tumor analysis pipeline. The full MRI scan is first processed by a YOLO-based RepVGG-GELAN detector to identify and crop padded tumor regions. These cropped regions are then passed to an EfficientUNet++ segmentation model to generate fine-grained binary masks, which are resized and mapped back to the original image space for visualization.

4.2 Tumor Detection using RepVGG-GELAN

The first stage of the pipeline employs the RepVGG-GELAN-C architecture, a YOLO-style object detector proposed by Balakrishnan and Sengar (2024), specifically designed to provide high-speed, high-accuracy detection for small objects in medical images. The model was implemented without any architectural modifications, using the configuration defined in models/detect/rcs-gelan-c.yaml. Training was conducted from scratch on the tumor dataset without pretrained weights.

4.2.1 Model Overview

RepVGG-GELAN combines two core components:

- **RepVGG Backbone**:
  A reparameterized convolutional architecture that uses VGG-style plain CNN blocks during inference. During training, it retains batch normalization and identity connections, which are fused into the convolutional layers before deployment to reduce inference overhead. This enables the model to maintain residual learning advantages during training while achieving fast single-path inference.

- **GELAN Detection Head**:
  An advanced YOLO detection module integrating the ELAN (Efficient Layer Aggregation Network) design. It focuses on reducing information loss during deep feature fusion and ensures efficient multi-scale processing across detection heads.

4.2.2 Backbone Architecture Details

The backbone consists of:

- 4 downsampling stages, each reducing spatial resolution by strided convolutions.
- 3 RepNCSPELAN4 modules, each combining:
  - Convolutional layers with different dilation rates and kernel sizes
  - Split-transform-merge paths for efficient channel mixing
  - Residual shortcuts (reparameterized at inference)
- ADown layers that combine average pooling with learnable convolutional paths for smoother downsampling.

The final output from the backbone feeds into the neck (aggregation layers) and head.

4.2.3 Detection Head and Output

The detection head operates on three feature maps:

- P3 (high-resolution, small tumors)
- P4 (mid-scale)
- P5 (low-resolution, large tumors)

These multi-resolution feature maps are processed using:

- **SPPELAN Module:** A spatial pyramid pooling variant that enhances contextual information by aggregating features at multiple receptive fields, improving detection across varying tumor sizes and shapes.
- **RepNCSPELAN4 Blocks:** These blocks further refine features using parallel convolutional paths and deep split-transform-merge fusion. Their design enables robust multi-scale feature representation and minimizes information loss during fusion. Unlike typical attention modules, these blocks use efficient residual structures with channel-wise partitioning to maintain lightweight computation.
- **DDetect Module:** The final detection module applies anchor-based convolutional layers to predict tumor locations. It outputs bounding box coordinates [x_center, y_center, width, height], an objectness score, and a class confidence score. Bias initialization in this module is dynamically adjusted based on expected object frequency and image scale, improving early training stability.

YOLO detection is used, and since the task is binary (tumor vs. background), the output tensor at each scale has a shape of [batch, anchors, 6], where 6 corresponds to [x, y, w, h, obj, class_confidence]. Each detection scale contributes predictions over its spatial grid, ensuring tumors of varying sizes are captured across P3, P4, and P5.

4.2.4 Detection-to-Segmentation Integration

To bridge detection and segmentation, crucial adaptations were made at the inference level. A custom logic was implemented in detect.py to:

- Apply a fixed padding margin to each detected bounding box (to preserve context)
- Crop the tumor region (detection) from the MRI scan
- Save the crop and its corresponding annotation (pulled from dataset) to disk for segmentation training
- Separate nested tumors into a separate folder as they don't need to by segmented

This cropping strategy was motivated by the need to avoid overfitting the segmentation model to unnaturally tight bounding boxes, which can go over tumor edges or overlook peritumoral tissue that is relevant for accurate segmentation.

During inference, confidence thresholding and non-maximum suppression (NMS) are applied to filter out redundant or low-confidence boxes. The NMS IoU threshold and object confidence were tuned via validation.

4.2.5 Model Deployment and Output Format

Input size: 640×640 pixels

Detection scales: The detection head processes three feature maps at progressively lower resolutions (P3, P4, and P5) which enable the model to detect tumors at small, medium, and large scales respectively. These feature maps are used internally to generate bounding box predictions across multiple anchor locations.

Bounding box format: Bounding boxes are predicted in the [x_center, y_center, width, height] format. These predictions are passed through non-maximum suppression and confidence thresholding.

Output format: For downstream segmentation, each selected bounding box is padded and used to extract a cropped image region from the original-resolution MRI. These image crops are saved as inputs to the EfficientUNet++ model.

Cropped region resolution: Preserved from the original MRI scan dimensions; not resized at this stage.

Inference time: 0.0ms pre-process, 86.4ms inference, 165.6ms NMS per image at shape (1, 3, 640, 640)

This detection stage provides the spatial grounding for the segmentation model. Its use of multi-scale anchors, reparameterized convolution, and context-preserving padding ensures that even small or irregular tumor regions are localized with minimal computational overhead.

4.3 Segmentation using EfficientUNet++

The second stage of the pipeline is responsible for producing precise, pixel-level segmentation of brain tumor regions identified in the detection phase. This task is handled by a custom-built EfficientUNet++ model, which combines the encoder efficiency of EfficientNet-B0 with the densely connected decoder design of U-Net++. The architecture is specifically chosen to optimize performance on limited medical datasets, where capturing fine boundary details and structural context is critical.

The segmentation model receives as input the padded tumor region crops extracted by the YOLO detector. Each crop is paired with a corresponding binary mask derived from the annotation in the dataset, resized to a standard input shape of 256×256 pixels. During inference, the model outputs a binary segmentation mask of the same resolution, identifying tumor pixels within the crop.

4.3.1 Encoder: EfficientNet-B0

The encoder in the EfficientUNet++ architecture is based on EfficientNet-B0, implemented using the timm library and initialized with pretrained ImageNet weights. EfficientNet-B0 is chosen for its lightweight design and its ability to extract hierarchical features using a compound scaling strategy, which balances between depth, width, and resolution to balance accuracy and efficiency.

Initializing the model, EfficientNet-B0's convolutional blocks are loaded and truncated at key points to extract intermediate feature maps, which are used as skip connections in the decoder. This process is handled by a custom class Encoder defined in unet.py.

The following components are extracted from the EfficientNet-B0 model:

- **stem**: Initial convolutional layer and batch normalization.
- **stages 0–3**: Sequential blocks from the EfficientNet model, grouped as:
    - stage0: First Mobile Inverted Bottleneck (MBConv) block
    - stage1: Blocks up to the first spatial downsampling
    - stage2: Deeper blocks providing mid-level semantic features
    - stage3: The deepest block, providing high-level abstracted features

The encoder outputs a list of feature maps at four spatial resolutions:

Table 1: Output feature maps from the EfficientNet-B0 encoder used in EfficientUNet++. Each stage corresponds to progressively deeper layers of the encoder, capturing features at increasing semantic levels and decreasing spatial resolution. These multi-scale outputs are used as skip connections in the decoder.

Stage	*Output Name*	*Feature Map Shape (Typical)*	*Channels*

Stem	features[0]	(B, 24, H/2, W/2)	24
Stage 0	features[1]	(B, 40, H/4, W/4)	40
Stage 1	features[2]	(B, 112, H/8, W/8)	112
Stage 2	features[3]	(B, 320, H/16, W/16)	320

These encoder outputs are passed to the decoder at corresponding levels for skip connections and feature fusion. The encoder does not freeze any layers, all weights are trainable, allowing fine-tuning of the pretrained features on our dataset.

The EfficientNet backbone includes depthwise separable convolutions, squeeze-and-excitation (SE) modules, and Swish activations internally; all of which are preserved and reused without alteration in this implementation.

The encoder's multi-resolution outputs serve two purposes:

1. They act as skip connections in the decoder to preserve spatial detail.
2. They provide hierarchical semantic context for dense segmentation.

4.3.2 Decoder: Nested U-Net++ Architecture

The decoder follows a U-Net++ decoder design, characterized by nested dense skip connections and multi-level feature aggregation. This architecture is intended to bridge the semantic gap between encoder and decoder features by enabling progressive refinement of spatial and contextual information through interleaved decoder blocks.

4.3.2.1 Nested Skip Pathways

Compared to the original U-Net, which links encoder and decoder layers only at matching depths, U-Net++ introduces a grid-like connection scheme. In this implementation, the decoder builds intermediate nodes such as x0_1, x0_2, x0_3, x1_1, and x1_2 by combining:

- Shallow encoder outputs
- Previous decoder outputs at the same level
- Upsampled features from deeper decoder layers

Each block includes:

- A ConvTranspose2d upsampling step
- Concatenation of inputs
- A Conv2d → BatchNorm2d → ReLU stack to refine fused features

These skip pathways allow the model to learn spatial detail and contextual meaning more gradually, improving segmentation of irregular or ambiguous tumor regions.

4.3.2.2 Fusion Blocks

Dedicated convolutional fusion modules (conv_fuse1, conv_fuse2, conv_fuse3, conv_fuse4) merge multiple intermediate outputs at each nested stage, allowing the decoder to blend shallow and deep features effectively. This modular fusion improves both accuracy and training stability, especially on small datasets.

4.3.2.3 Absence of Deep Supervision

Unlike the full U-Net++ architecture described by Zhou et al. (2018), which includes side-output classifiers (deep supervision) at multiple intermediate decoder nodes, this implementation uses only a single segmentation head applied to the final output node x0_3. There are no auxiliary outputs or intermediate loss branches. This simplifies training and reduces the parameter count, while still retaining the core benefit of dense feature fusion across scales. Deep supervision was implemented earlier and the model was trained; however, no performance improvement was observed. Therefore, the final model did not include it to maintain architectural simplicity and computational efficiency.

4.3.2.4 Decoder Summary

This U-Net++-inspired decoder preserves the most critical structural innovation (dense skip connections and nested refinement blocks) while omitting the deep supervision components. It provides a strong balance between architectural depth and training complexity, allowing accurate segmentation of brain tumors while maintaining low computational overhead.

4.3.3 Output Layer and Thresholding

The final prediction of the segmentation mask is generated from the decoder node x0_3, which contains the most refined feature representation after successive nested fusions. This output is passed through a dedicated 1×1 convolutional layer to reduce the number of output channels to one, corresponding to a single-class (tumor vs. background) segmentation task.

The model applies the following steps:

4.3.3.1 1×1 Convolution

A final Conv2d(in_channels=filters, out_channels=1, kernel_size=1) reduces the channel depth from the last decoder output down to a single-channel output, where each pixel represents the model's confidence of tumor presence.

The shape of the output tensor is (B, 1, 256, 256), corresponding to the batch size, single channel (binary classification), and spatial dimensions of the input crop.

4.3.3.2 Sigmoid Activation

The output logits are passed through a Sigmoid() activation function. This compresses the output values into the [0, 1] range, producing a soft probability map where each pixel value indicates the likelihood that it belongs to the tumor region.

4.3.3.3 Thresholding at Inference Time

The model's probability output is threshold ed at 0.5 to produce a binary mask:

- Pixel values ≥ 0.5 are classified as tumor (foreground).

- Pixel values < 0.5 are classified as background.

This results in a binary mask of the same spatial size as the input crop (256×256), where each pixel denotes the predicted tumor region.

4.3.3.4 Output Format

The thresholded output is used in two ways depending on the script context:

- **During training and validation**:
  - The predictions are compared against ground truth masks using Dice loss and BCE loss.
  - Visual outputs (predictions_epoch.png) are saved each epoch showing multiple samples (prediction vs. ground truth) for manual inspection.
- **During inference**:
  - The binary mask is overlaid on the original MRI crop using Jet colormap.
  - The mask is saved as .png in the results folder.
  - No post-processing or resizing is done, the mask retains the 256×256 resolution of the padded input crop.

The inference script stitches the predicted masks back onto the original full-size MRI canvas for visualization purposes, but the segmentation itself is strictly confined to the 256×256 detection-aligned patch.

4.3.4 Data Flow

The data flow through the segmentation pipeline is structured and consistent across training, validation, and inference. Each step ensures alignment between input image crops and their corresponding tumor masks, preserving spatial consistency for accurate segmentation.

4.3.4.1 Input Format

The input to the EfficientUNet++ model is a 256×256 grayscale image, which is a padded crop extracted from the original MRI using the bounding box predicted by the detection model. This cropping process is handled by detect.py, which ensures:

- The tumor is centered within the crop.
- A fixed padding is applied to include surrounding context.
- Cropped images and masks are saved with matching filenames.

All inputs are read as one channel grayscale images and expanded to match the expected input shape (B, 1, 256, 256).

4.3.4.2 Training Pipeline

1. **Dataset Class**:
   A custom dataset class SegmentationDataset (in train_UNet.py) is used to load the image–mask pairs. Each sample consists of:
   - One grayscale image (padded crop)
   - One binary mask (same shape, tumor = 255, background = 0)
2. **Joint Augmentations**:
   The dataset class applies synchronized augmentations to both the input image and its corresponding mask using the JointTransform module:
   - Random horizontal/vertical flips
   - Random rotations (±15 degrees)
   - Color jittering (contrast, hue, brightness)
3. **Normalization**:
   - Images are normalized to [0, 1] using ToTensor().
   - Masks are binarized and scaled from [0, 255] → [0, 1].
4. **Forward Pass**:
   - Input is passed through EfficientUNet++.
   - Output: A single-channel sigmoid map of shape (B, 1, 256, 256).
5. **Loss Computation**:
   - BCE + Dice loss is computed using the predicted mask vs. ground truth.
   - Metrics (Dice, IoU, accuracy, etc.) are computed and logged.
6. **Outputs**:
   - predictions_epoch.png is saved per epoch showing prediction vs. ground truth.
   - All metric values (train/val loss, Dice, IoU, etc.) are written to CSV.

4.3.4.3 Validation Flow

- Identical to training, but:
  - No data augmentation is applied.
  - Model is run in evaluation mode (model.eval()).
  - Thresholding at 0.5 is applied to convert sigmoid output to binary mask.
  - Results are logged and visualized.

4.4 Training

The two models were trained separately using two different training scripts, one training script for object detection using the RepVGG-GELAN model, and the other for semantic segmentation using the EfficientUNet++ model. Each model was trained independently to ensure optimal performance for its specific task, using tailored loss functions, data augmentation, and learning rate schedules.

4.4.1 Detection Model Training (RepVGG-GELAN)

The YOLO-based detection model was trained using a modified implementation of the RepVGG-GELAN architecture, as defined in the source paper by Balakrishnan (2024). The training was performed using the **train.py** script adapted from the original repository.

Training components:

- **Loss Function**: A multi-component loss comprising bounding box regression (CIoU), objectness, and classification losses.
- **Hyperparameters**: Loaded from a dedicated hyp.yaml file, with key settings for learning rate, momentum, weight decay, and IoU thresholds.
- **Optimizer & Scheduler**: Smart optimizer is used with support for SGD, Adam, AdamW or LION with various learning rate strategies including cosine decay, flat cosine, and fixed schedules.
- **Data Loader**: The dataloader supports image augmentation (scaling, flipping, mosaic), label smoothing, and anchor checking.
- **Training Epochs**: Default of 100 epochs, can be changed via arguments, with **early stopping** and **model checkpointing** based on validation metrics.
- **Evaluation Metrics**: **Precision**, **Recall**, **mAP@0.5**, and **mAP@0.5:0.95** on validation data after each epoch.

4.4.2 Segmentation Model Training (EfficientUNet++)

The segmentation model was trained using a custom script **train_UNet.py**, which leverages an EfficientUNet++ architecture with an EfficientNet-B0 encoder and deeply nested decoder pathways.

Training components:

- **Loss Functions**:
    - Binary Cross-Entropy (BCE) for per-pixel classification.
    - Dice Loss for overlap-based segmentation accuracy.
    - A weighted combination was used: Loss = BCE + $\alpha \times$ Dice, where $\alpha = 5$.
- **Augmentation**: Applied jointly to images and masks, including:

    - Random flips (horizontal and vertical)
    - Random rotations (up to 15 degrees)
    - Color jitter (brightness, contrast, hue)
- **Warmup Strategy**: Learning rate was linearly increased for the first few epochs, followed by a CosineAnnealingLR schedule.
- **Early Stopping**: Implemented with a patience of 5 epochs to halt training if validation loss is not improving.
- **Validation**:
    - Computing and logging Loss, Dice, IoU, Pixel Accuracy, Precision and Recall to TensorBoard.
    - Save a “predictions_epoch.png” in each epoch folder showing 4 random images with Ground Truth vs predicted masks for manual inspection.
- **Logging & Checkpointing**:
    - Models and logs were saved per experiment under runsUNet/train/exp*.
    - Metrics including loss, Dice score, and LR were saved to CSV and plain text for reporting.
- **Final Output**: The best-performing model based on validation loss and the last epoch model are both saved.

4.5 Data Preparation

This project uses the Brain Tumor Detection dataset by Ahmed Hamada, which is a publicly available dataset on Kaggle. It contains grayscale 2D MRI brain scans with tumor annotations provided in JSON format. The dataset is pre-split into training, validation, and test sets, containing 500, 200, and 100 images respectively.

4.5.1 Detection Pipeline Preparation

To train the YOLO-based detection model, a custom script (converter.py) was used to convert annotations into YOLO format. For each image:

- Tumor regions were extracted from the annotation JSON.
- Bounding boxes were computed and normalized to the YOLO format [class_id, x_center, y_center, width, height].
- Tumors were labeled under a single class 0.

The dataset was already split into training, validation, and test folders. These were used directly during training without further reshuffling. During inference, the detect.py script runs the trained YOLO model, automatically applying the configured padding to each detection. This script:

- Applies automatic padding around detected tumor regions,
- Outputs cropped tumor patches from the original images,
- Generates corresponding binary mask crops based on the original annotations, and
- Separates nested tumors into a dedicated folder for optional further processing.

These padded image crops and their corresponding masks serve as the direct input for the segmentation model.

4.5.2 Segmentation Pipeline Preparation

To prepare the dataset for EfficientUNet++ training, additional processing was necessary:

- The split_annotations.py script was used to separate the aggregated JSON file into individual JSON annotation files per image, allowing easier lookup during segmentation mask generation.
- The detect.py script, run in segmentation mode, also generated **binary segmentation masks** corresponding to each cropped image. These masks highlight tumor areas in white (pixel value 255) on a black background (0).

All cropped tumor images and their masks were resized to 256×256 pixels to fit the input size expected by the segmentation network. Data was loaded using a custom PyTorch SegmentationDataset class, which ensured that each image-mask pair was aligned and allowed for joint augmentations using a JointTransform module.

Only basic preprocessing was applied: intensity normalization to a [0, 1] range, resizing, random flips, rotations, and color jitter for robustness. No domain-specific preprocessing (e.g. contrast equalization) was applied.

4.5.3 Annotation Filtering and Quality Control

During testing and data preparation, it was observed that some annotations in the dataset are defined as ellipses or circles, which made the annotations not align accurately with the tumor boundaries. These imprecise annotations in the dataset could negatively affect our model's performance.

To address this, the detect.py script was modified and ran with the --exclude-circles and --exclude-ellipses flags enabled. This excluded all non-polygon annotations, ensuring that only high-quality polygon-based tumor outlines were used during segmentation training. In my experiments, excluding these low-quality annotations resulted in a 1.5% improvement in Dice score, confirming their negative impact on model performance.

4.6 Inference

After training, a dedicated inference script (inference.py) was used to run the final two-stage model on test images. This script integrates both the trained YOLO-based RepVGG-GELAN detector and the EfficientUNet++ segmentation model into a seamless pipeline.

For each input image, the following steps are performed:

1. **Tumor Detection (YOLO)**
    - The trained YOLO model is loaded and used to predict bounding boxes for tumor regions.
    - Predictions are filtered using confidence and IoU thresholds, followed by non-max suppression.
    - Detected bounding boxes are then padded (default: 10 pixels) to include contextual information around the tumor.
    - Each padded region is cropped from the original image for segmentation.
2. **Tumor Segmentation (EfficientUNet++)**
    - Each cropped region is resized to 256×256 and passed to the trained EfficientUNet++ model.
    - The model outputs a probability map, which is thresholded (default: 0.5) to create a binary segmentation mask.
    - This mask is resized back to the original cropped region size and inserted into a full-size empty mask aligned with the input image.
3. **Visualization and Output**
    - The original image, the image with YOLO-drawn boxes, and the image with segmentation masks overlaid are combined into a single side-by-side visual.
    - All outputs are saved in a dedicated folder under runs/ with filenames reflecting the input image.
    - The user can view the combined visualization window and inspect results interactively.

This post-training inference script reflects the complete deployment path for the model: from full-size image to bounding box detection, to segmentation, and finally to visual output. Notably, nested tumors are not segmented by EfficientUNet++, they are detected and saved during preprocessing but are excluded during this final inference stage.

4.6.1 Inference Run

To illustrate the complete inference workflow, an example run is presented below. The figure demonstrates the output of the two-stage pipeline when applied to a single MRI scan from the test set.

- The original image is shown on the left.
- The YOLO-detected region is shown in the middle, with the bounding box and padding applied.

- The final segmented tumor mask, produced by EfficientUNet++, is shown on the right, overlaid on the input crop.

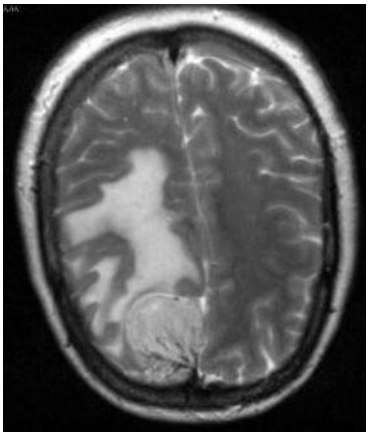

Figure 4: Original input brain MRI scan. No annotations or predictions are shown.

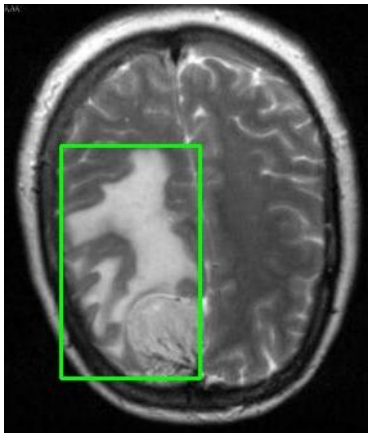

Figure 5: YOLO-based tumor detection result. The green bounding box indicates the predicted tumor region with added padding.

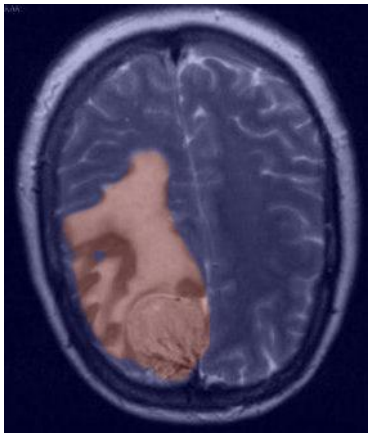

Figure 6: Final segmentation output from EfficientUNet++, showing the predicted tumor mask overlaid as a color map on the padded crop.

5. Implementation

This section outlines the technical setup and practical details required to replicate the detection and segmentation pipeline developed for this project. It includes environment configuration, dataset handling, model structure, and training workflow.

5.1 Environment

The models were implemented using Python 3.9.13 and PyTorch deep learning framework version 2.6.0 paired with CUDA 12.6. All experiments were conducted on a machine equipped with an NVIDIA GeForce RTX 3070 Laptop GPU with 8GB VRAM.

5.2 Project Structure

The implementation was modular and separated into detection and segmentation stages, allowing both to be run independently or as a complete pipeline. Key files:

```
RGELAN/
 ├── dataset/
 ├── models/
 ├── runs/
 ├── runsUNet/
 ├── checkDevices.py
 ├── command.txt
 ├── detect.py
 ├── inference.py
 ├── train.py
 ├── train_UNet.py
 ├── val.py
 └── val_UNet.py
```

5.3 Dataset Preprocessing

The BR35H dataset was used as the primary source of MRI scans and tumor annotations. Two key scripts were developed to preprocess the dataset separately for the detection and segmentation stages of the pipeline:

- **For the detection model (YOLO – RepVGG-GELAN)**, a script named converter.py was used. This script converts the original annotation format from annotations.json into YOLO-compatible bounding box annotations. It supports various annotation shapes, including polygons, ellipses, and circles, by converting them into bounding boxes. Each annotation is saved in a separate .txt file corresponding to the image, formatted according to YOLO standards.
- **For the segmentation model (EfficientUNet++)**, a separate script called split_annotations.py was used to divide the full annotations.json file into individual JSON files for each image. This made it easier to manage, visualize, and prepare image-mask pairs.

The exclusion of circular and elliptical annotations was not done during dataset preprocessing but as part of the detection-to-segmentation bridge within the detect.py script. In detect.py, only tumors annotated as polygons were forwarded to the segmentation model. This filtering step improved the quality of segmentation training by ensuring that only precise, edge-aligned tumor masks were used.

5.4 Training Setup

The training process was divided into two main stages: tumor detection and tumor segmentation. Each stage had its own dataset preparation flow, architecture, hyperparameters, and training logic.

5.5.1 Detection Model – RepVGG-GELAN (YOLO-based)

The tumor detection model was trained using a RepVGG-GELAN configuration designed for object detection tasks. The training was initiated using the following command:

```
python train.py --img 640 --batch 8 --epochs 150 --min-items 0 --close-mosaic 15 --data coco.yaml --
cfg models/detect/rcs-gelan-c.yaml --hyp data/hyps/hypscratch-high.yaml --device 0
```

Key details:

- **Image size**: 640×640
- **Epochs**: 150
- **Batch size**: 8
- **Learning rate**: 0.01 (with final LR = 0.0001 via OneCycleLR)
- **Optimizer**: SGD with momentum 0.937 and weight decay 0.0005
- **Warmup**: 3 epochs with bias LR warmup = 0.1
- **Augmentations**:
    - Mosaic (enabled for first 135 epochs)
    - Mixup: 0.15
    - Copy-paste: 0.3
    - Flip LR: 50%, Translation: ±10%, Scale: ±90%, HSV jitter (H: 0.015, S: 0.7, V: 0.4)

Loss components:

- Box loss gain: 7.5
- Objectness loss gain: 0.7
- Classification loss gain: 0.5
- DFL loss gain: 1.5
- IoU threshold: 0.20

Dataset details:

- Defined via coco.yaml, using a custom dataset with a single class: "brain tumor".
- Paths for training and validation are manually set to the BR35H detection folders.

5.5.2 Segmentation Model – EfficientUNet++

Following detection, the segmented tumor region (cropped and padded by detect.py) was passed to the segmentation model. This model is a custom EfficientUNet++ with nested skip connections and an EfficientNet-B0 encoder. Training was launched with:

```
python train_UNet.py --images_dir dataset\UNet\TRAIN2\crops --masks_dir dataset\UNet\TRAIN2\crop_masks
--val_images_dir dataset\UNet\VAL2\crops --val_masks_dir dataset\UNet\VAL2\crop_masks
```

Key details:

- **Input size**: 256×256
- **Epochs**: 50
- **Optimizer**: Adam with weight decay = 0.0001
- **Learning rate**: 0.0005
- **Loss function**: Combination of Binary Cross-Entropy (BCE) and Dice loss, with Dice loss weighted 5× more than BCE to emphasize spatial overlap accuracy

Augmentation settings:

- Horizontal flip: 50%
- Vertical flip: 20%
- Random rotation: ±15°
- Color jitter:
    - Brightness, contrast, saturation: 0.2
    - Hue shift: 0.1

Both training and validation outputs (including predicted masks, overlays, and metrics like Dice and IoU) were saved for each epoch to allow for visual and numerical inspection.

6. Results and Evaluation

This section presents the quantitative and qualitative results of the two-stage brain tumor detection and segmentation pipeline. The performance of the YOLO-based detection model and the EfficientUNet++ segmentation model were evaluated separately using appropriate metrics.

6.1 Detection Results (YOLO – RepVGG-GELAN)

The tumor localization component of this project was implemented using the RepVGG-GELAN model, a YOLO-style detector introduced by Balakrishnan and Sengar (2024). The model was trained from scratch. The results obtained from this implementation are closely aligned with the performance reported in the original study, with only minor deviations observed, likely attributable to random initialization.

A comparison between the performance metrics reported in the original RepVGG-GELAN paper and those obtained in this implementation is shown in Table 5.1. The comparison

includes key indicators such as precision, recall, and mean average precision at different IoU thresholds.

Table 2: Performance comparison between RepVGG-GELAN (original paper) and this project's implementation.

	Precision	*Recall*	*AP50*	*AP50:95*
Paper Result	0.982	0.890	0.970	0.723
Our Result	0.986	0.907	0.959	0.730

6.1.1 Training Diagnostics

To better understand the model's learning behavior, a series of diagnostic plots were automatically generated throughout training using the built-in utilities of the YOLO training script. These plots provide insights into class distribution, label quality, and detection calibration over varying thresholds.

6.1.1.1 Label Distribution

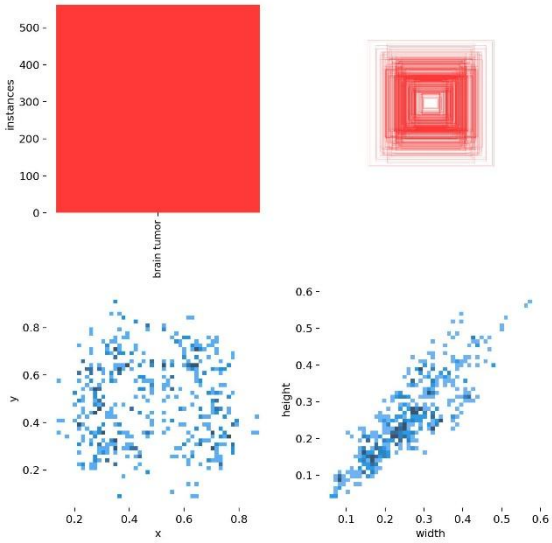

Figure 7: Displays the frequency of bounding box instances across the training set.

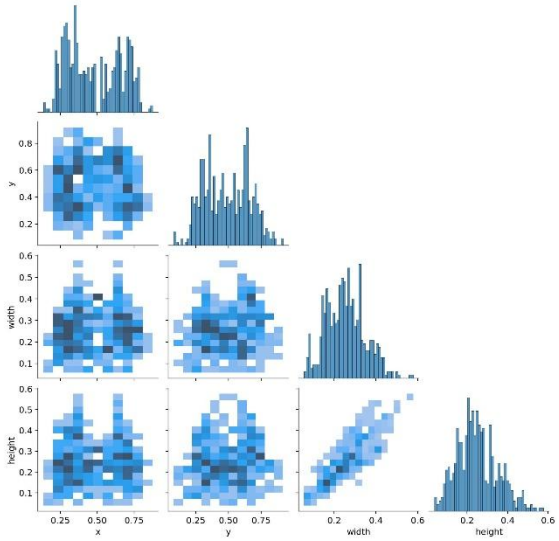

Figure 8: Although more relevant for multi-class datasets, this plot shows co-occurrence relationships between labels.

6.1.1.2 Model Calibration and Confidence Analysis

The following curves illustrate how the model balances confidence, precision, and recall across different IoU thresholds:

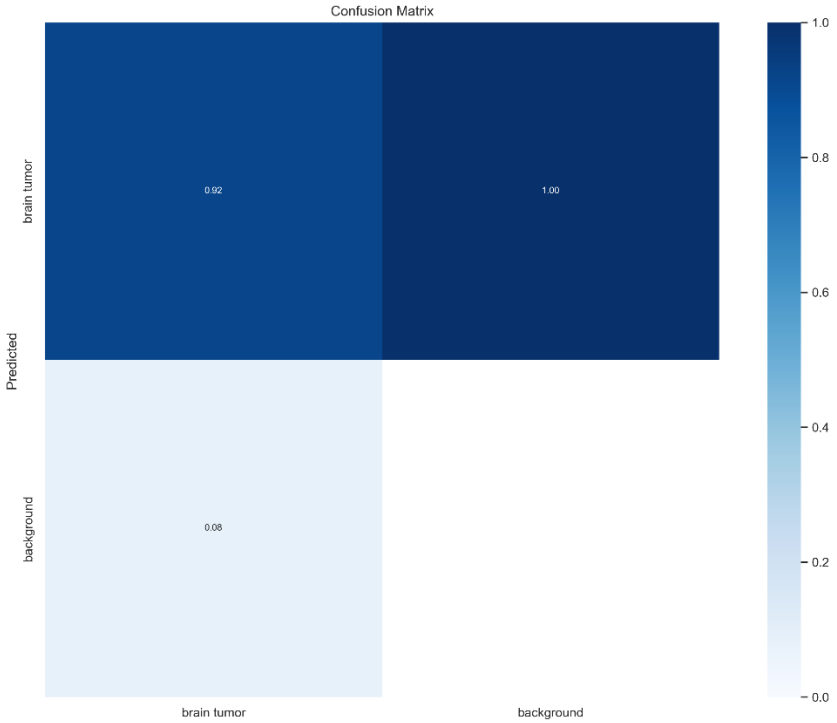

Figure 9: Shows the balance between true positives and false detections on the validation set.

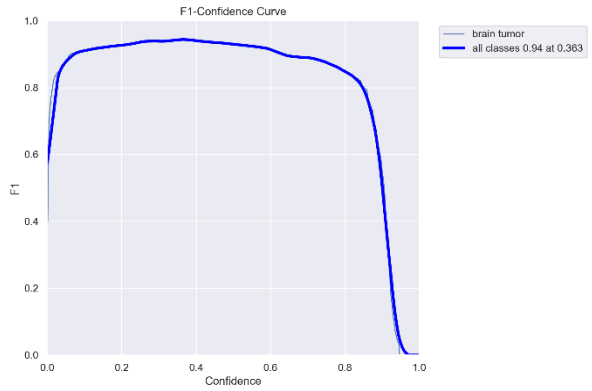

Figure 10: Displays the F1 score evolution across thresholds, helping identify an optimal operating point that balances precision and recall.

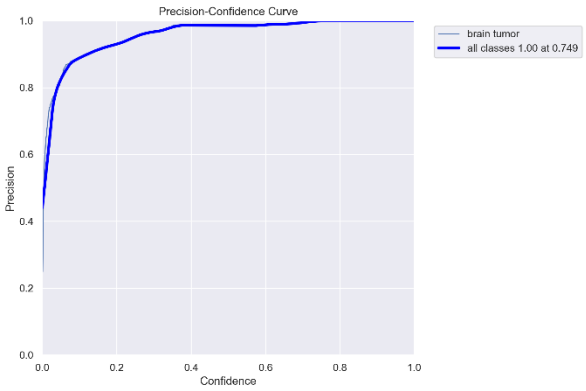

Figure 11: Plots precision across various confidence thresholds.

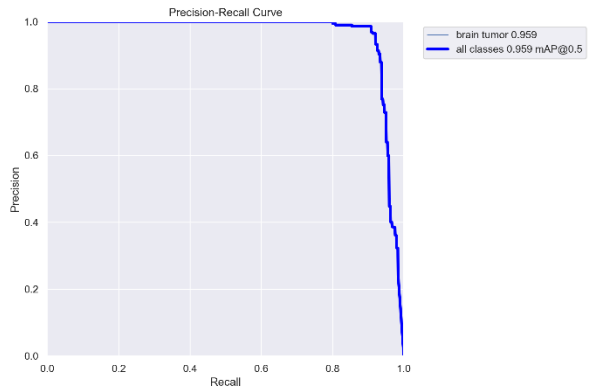

Figure 12: Combines precision and recall into a single curve, providing a holistic view of detection trade-offs.

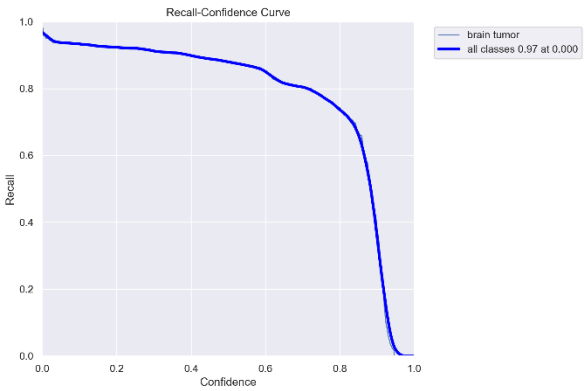

Figure 13: Recall curve indicating how well the model detects all instances across thresholds.

6.2 Segmentation Results (EfficientUNet++)

The segmentation stage of the pipeline was evaluated using the evaluation set of 133 cropped tumor regions. The model used was EfficientUNet++, trained on padded YOLO-detected crops resized to 256×256 pixels. Segmentation performance was assessed using common pixel-level and region-level metrics, including the Dice Similarity Coefficient (DSC), Intersection over Union (IoU), pixel-wise precision and recall, and overall accuracy.

Table 3: Segmentation performance metrics on the test set.

METRIC	**VALUE**
DICE SCORE (DSC)	0.9375
INTERSECTION OVER UNION (IOU)	0.8944
PRECISION	0.9509
RECALL	0.9592
PIXEL ACCURACY	0.9372

6.2.1 Training Trends and Metric Evolution

To analyze how the model learned over time, key metrics were tracked across epochs. These include:

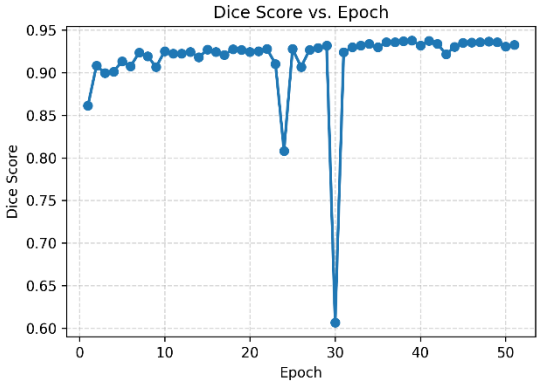

Figure 14: Line graph of Dice score vs. Epoch, showing the model's segmentation performance improving over time.

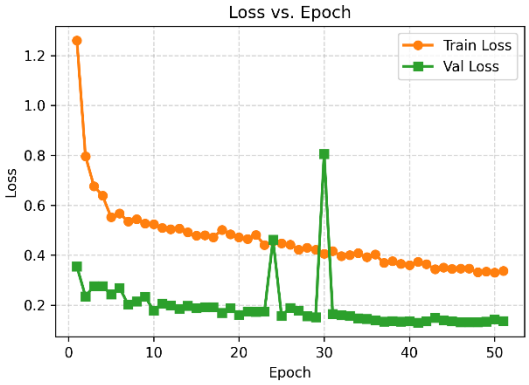

Figure 15: Training and validation loss curves, reflecting learning stability and generalization. A gap between the two would indicate overfitting; consistent trajectories suggest good model tuning.

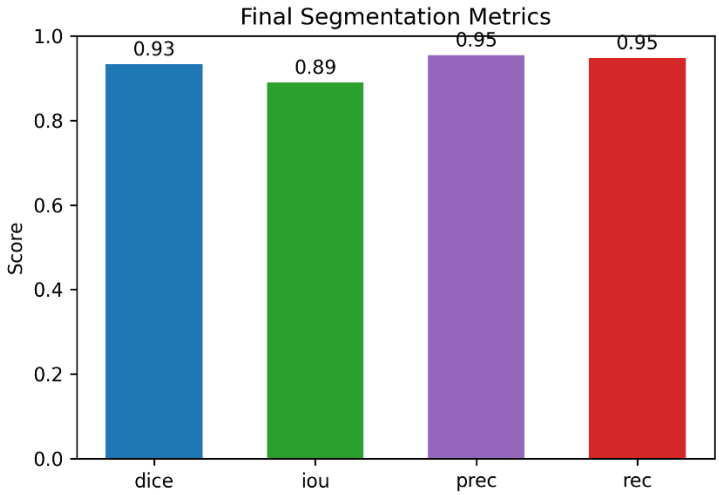

Figure 16: A bar chart comparing final validation metrics—Dice, IoU, precision, and recall—at the best-performing epoch.

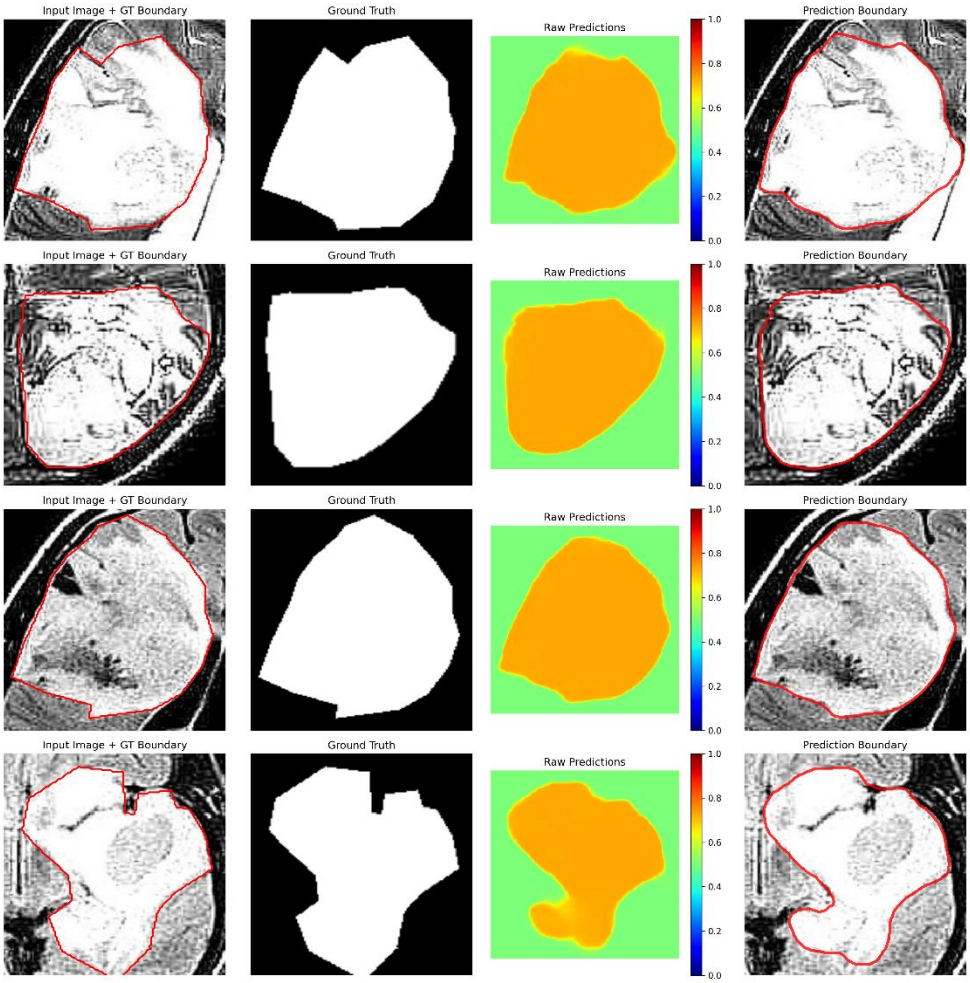

Figure 17: Sample visualization, displaying input images, ground truth masks, and corresponding predicted outputs. This gives a qualitative view of how well the model captures tumor boundaries.

These figures were generated using TensorBoard logs and the metrics.csv file created during training. Together, they demonstrate stable model learning and consistent segmentation performance across validation batches.

7. Design Justification

The selection of model architectures in this project was steered by the specific demands of brain tumor analysis from MRI scans: accurate localization, fine-grained boundary segmentation, computational efficiency, and generalization capability under limited annotated data.

7.1 Two-Stage Pipeline

Instead of adopting an end-to-end instance segmentation framework, this project utilizes a decoupled two-stage pipeline consisting of object detection followed by semantic segmentation. This decision was made to leverage the individual strengths of specialized models: fast and robust region-of-interest (ROI) localization using YOLO-based detection, and high-resolution boundary delineation using an encoder-decoder segmentation network. This

modular design provides better and easier debugging and optimization for each task, especially in a medical context where explainability and performance tuning are essential.

7.2 RepVGG-GELAN

RepVGG-GELAN was selected as the detection backbone due to its balance of detection accuracy, lightweight architecture, and fast inference speed. As a recent advancement in the YOLO family, GELAN improves feature aggregation across scales through the Efficient Layer Aggregation Network, while RepVGG provides a reparameterized convolutional structure that simplifies the inference graph after training. This combination allows the detector to handle tumors of varying sizes effectively with minimal computational overhead.

7.3 EfficientUNet++

The segmentation stage uses a custom EfficientUNet++ model, which integrates EfficientNet-B0 as the encoder with a U-Net++ nested skip-connected decoder. This architecture was selected after evaluation of its advantages in medical image segmentation tasks:

- **EfficientNet-B0** provides high-quality multi-scale features with significantly fewer parameters compared to traditional encoders like ResNet or VGG. Its compound scaling strategy ensures that width, depth, and resolution are increased in a balanced way, improving representation power while maintaining efficiency.
- **U-Net++ decoder** incorporates dense skip connections and interleaved refinement paths that reduce the semantic gap between encoder and decoder layers. This allows the network to progressively refine boundary details, especially beneficial for segmenting tumors with irregular or fuzzy contours.
- **Modular design** of EfficientUNet++ supports flexible tuning and adaptation. For this project, deep supervision was omitted to reduce model complexity, but the architecture retains its structural benefits through its nested pathways and multi-level feature aggregation.

This combination of lightweight and robust feature extraction, along with refined spatial fusion, positions EfficientUNet++ as a suitable segmentation model under both data and resource constraints.

7.4 Summary of Architectural Decisions

Table 4: Summary of key architectural components and the rationale behind their selection. Each model was chosen to balance accuracy, efficiency, and modularity within a two-stage brain tumor detection and segmentation pipeline.

COMPONENT	MODEL USED	RATIONALE
DETECTION	RepVGG-GELAN	Lightweight YOLO-based architecture with efficient multi-scale detection
SEGMENTATION	EfficientUNet++	High-resolution nested decoding with EfficientNet-B0 encoder
PIPELINE DESIGN	Two-stage approach	Separates localization and segmentation for better modularity and tuning

In summary, the architectures used in this project were selected based on their proven capabilities in handling high-resolution medical images and their computational efficiency. This structure also facilitates future enhancements or substitutions of individual components without reworking the entire system.

8. Discussion

This project explored a two-stage deep learning pipeline combining RepVGG-GELAN for brain tumor detection and EfficientUNet++ for segmentation. The overall system achieved strong results, both quantitatively and qualitatively, but its true value lies not only in raw metrics but in how those results translate into practical insights, design implications, and potential clinical relevance.

8.1 Interpretation of Results

The model showed strong performance across both tasks. Detection was robust in identifying most tumors with irregular boundaries, benefiting from GELAN's multi-scale feature handling and RepVGG's efficient architecture. However, detection still faltered in a few cases involving very small or faint tumors, where boundary contrast was too weak or the tumor was partially obscured by similar intensities in surrounding tissues.

Segmentation performance was similarly strong, with EfficientUNet++ producing high Dice and IoU scores. The nested skip connections and EfficientNet encoder enabled rich spatial feature extraction and accurate mask generation. Nonetheless, performance decreased slightly in cases with low-contrast or ambiguous boundaries, where the tumor blended into surrounding tissue.

These results suggest that while the system performs well in general, edge cases, particularly those involving subtle intensity variations, still challenge both detection and segmentation. These edge cases are often the most clinically important, especially in early diagnosis.

8.2 Architectural Trade-offs and Dataset Decisions

A core design decision was to implement a modular, two-stage pipeline rather than a single unified model. This allowed independent optimization of detection and segmentation tasks, making it easier to diagnose model weaknesses and fine-tune components. However, it also introduced a dependency: if the detection stage fails, the segmentation model receives incorrect or irrelevant input. This trade-off was accepted in favor of clearer modular control and explainability.

Another critical choice was the removal of circular and elliptical annotations from the dataset. This step reduced the number of usable samples but greatly improved the precision of the training labels. The segmentation model trained on the cleaned dataset produced masks that more accurately followed actual tumor boundaries and avoided learning artificial shapes caused by oversimplified annotations. This decision also reduced the tendency for the model to produce rounded segmentation outputs that did not reflect tumor morphology.

The selection of EfficientNet-B0 as the encoder was another pragmatic trade-off. While deeper variants could theoretically capture more complex features, B0 struck a strong balance between accuracy and speed, making it suitable for environments with limited computational resources.

8.3 Clinical Relevance and Real-World Potential

Beyond technical performance, this system shows promise for real-world clinical use. Its ability to automatically localize and segment brain tumors in MRI scans could assist radiologists by reducing the time needed for manual annotation and improving consistency in tumor analysis. The modular nature of the system also means that individual components (e.g., the detection module) could be deployed independently in simpler diagnostic workflows.

Still, real-world deployment would require further steps, including testing on more diverse MRI datasets, integration with radiology systems, and improvements in handling ambiguous cases. In particular, providing confidence scores or uncertainty maps at inference could help clinicians trust or question specific predictions.

8.4 Summary

In summary, the model met its objectives in delivering a lightweight yet accurate two-stage pipeline for brain tumor detection and segmentation. The results reflect not just successful training, but thoughtful architectural decisions and data handling. However, further work is needed to extend its usefulness to clinical settings, improve reliability in difficult cases, and continue closing the gap between algorithmic precision and practical healthcare support.

9. Limitations

Despite the results achieved by the proposed two-stage pipeline using RepVGG-GELAN for detection and EfficientUNet++ for segmentation, some limitations were observed during experimentation and result evaluation. These limitations span across both model-specific performance constraints and potential dataset-related inconsistencies.

9.1 YOLO Model Limitations

While RepVGG-GELAN demonstrated strong detection performance overall, it showed weaknesses in detecting small tumors, especially those with faint or poorly defined boundaries. Although the model handled irregular tumor shapes relatively well, its ability to localize subtle lesions with unclear contrast was limited. This is a known challenge in object detection models, where bounding boxes are often misaligned or missed entirely if the object lacks high-contrast features.

A clear example of this limitation is shown in Figures 18 and 19. The ground truth annotation highlights a small tumor region, but the model's prediction failed to detect any bounding box, completely missing the tumor. This suggests that YOLO-based detection, while effective, is less reliable for detecting subtle or ambiguous features in grayscale MRIs.

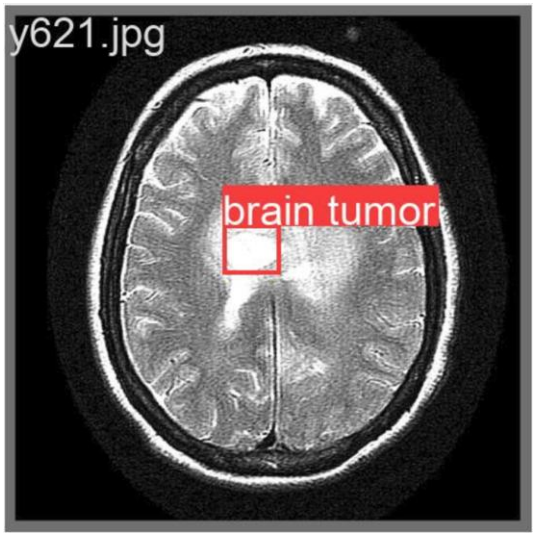

Figure 18: Ground truth bounding box highlighting a small tumor with low contrast.

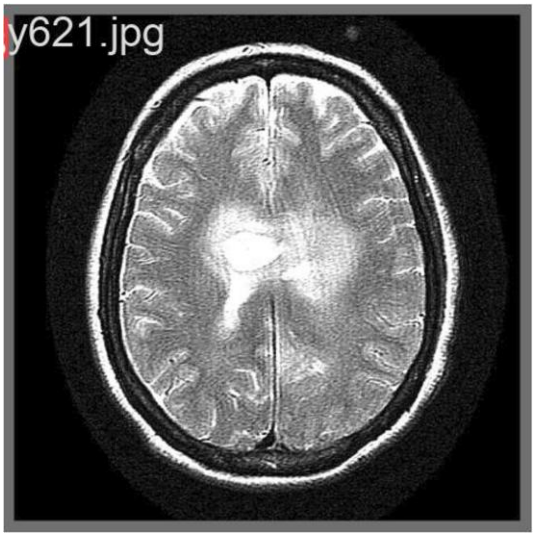

Figure 19: Model prediction showing no detection for the same tumor, despite its presence.

This behavior was not isolated and was observed in other similar cases throughout the dataset, suggesting a consistent detection gap for faint lesions. This limitation is critical in medical contexts where early-stage or small tumors with low signal intensity may be overlooked.

9.2 U-Net Segmentation Limitations

The EfficientUNet++ model did not suffer from the same size-related limitations as the detector because it received an already cropped region centered on the tumor. However, it too showed signs of struggling with faint or low-contrast tumor boundaries, especially when the edge transition between tumor and healthy tissue was subtle or ambiguous.

As shown in Figures 20 and 21, the ground truth mask correctly outlines a low-contrast tumor. However, the model's prediction under-segments the tumor, failing to capture its true shape. This discrepancy stems from the model's reduced sensitivity to subtle intensity transitions between tumor tissue and healthy tissue.

Figure 20: Ground truth mask accurately outlining a faint tumor region.

Figure 21: Predicted mask from EfficientUNet++, showing under-segmentation.

In such cases, the model occasionally produced over-segmented or under-segmented regions, which affected the Dice and IoU scores, despite strong performance on well-defined tumors. These segmentation errors were consistent with known limitations of encoder-decoder architectures when dealing with low-gradient regions in grayscale medical images.

9.3 Dataset Annotation Limitations

A further limitation lies in the quality of the BR35H dataset annotations. During evaluation, both the YOLO-based detector and the U-Net segmenter occasionally produced predictions that subjectively aligned better with the tumor region than the ground truth annotation. In such cases, the model highlighted plausible tumor areas that extended beyond or differed from the original label, suggesting that some annotations may be incomplete, imprecise, or overly simplified.

One clear example is shown in Figures 22 and 23. In Figure 22, the ground truth bounding box is much larger than the actual tumor, covering parts of the brain that do not contain any abnormality. On the other hand, the prediction made by the YOLO model in Figure 23 shows a smaller, more precise bounding box that actually fits the tumor better. Even though the model's prediction is more accurate, it is still marked as incorrect because it does not match the oversized label.

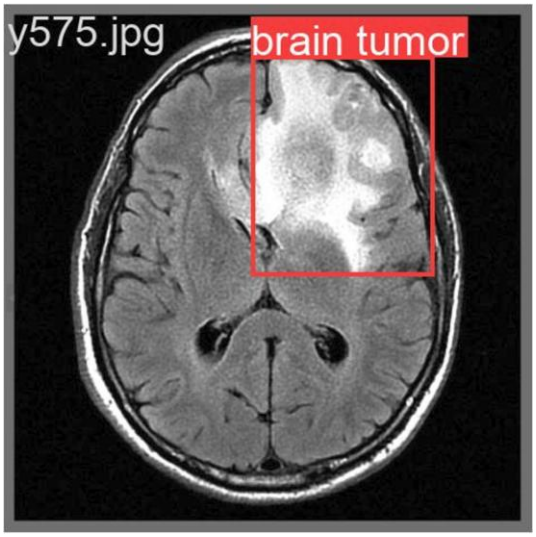

Figure 22: Ground truth bounding box, larger than the visible tumor area.

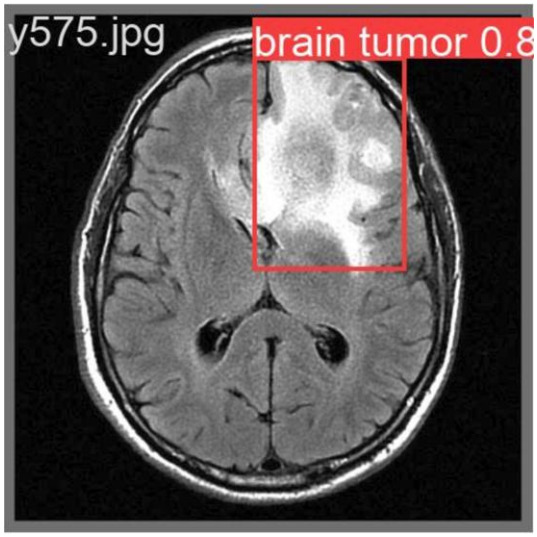

Figure 23: YOLO prediction with tighter and more accurate tumor coverage.

This kind of problem affects both training and evaluation:

- During training, the model learns from inaccurate examples, which can confuse it.
- During evaluation, a better prediction can still get a lower score if the ground truth is wrong.

Similar issues were seen with segmentation masks, particularly in how they were annotated. The BR35H dataset includes three types of annotation shapes: polygons, circles, and ellipses. Among these, polygons were the most accurate, closely following the actual edges of the tumors. However, circles and ellipses, which were used for tumors with a roughly round shape, often failed to match the true tumor boundary. No tumor is a perfect circle, so these types of annotations frequently cut into or extended beyond the actual tumor.

This inconsistency affected the model during training. The U-Net model learned from these circular annotations and sometimes began producing predictions that were also rounded, even when the tumor shape was irregular. As a result, the segmentation masks often showed a "circular effect", especially in cases where the ground truth label was oversimplified.

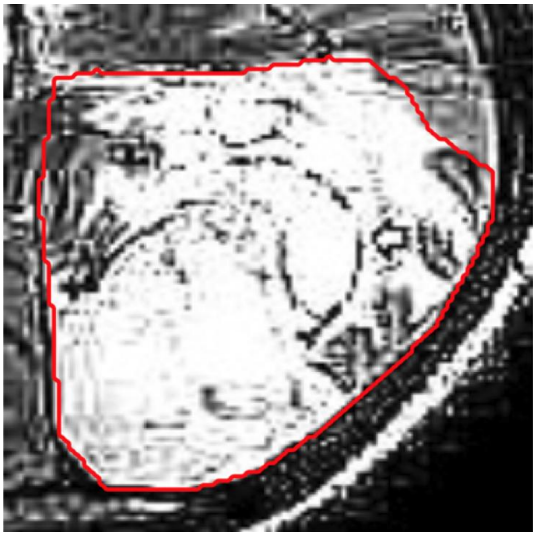

Figure 24: Ground Truth

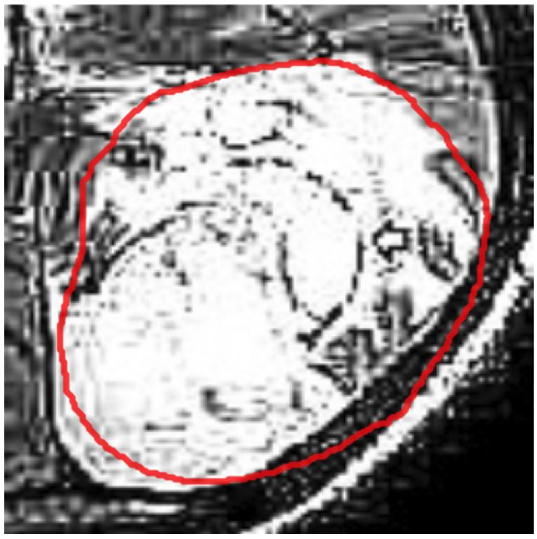

Figure 25: U-Net prediction showing a circular shape influenced by non-polygon ground truth annotation, despite the tumor having an irregular boundary.

Because of this issue, a dataset cleaning step was necessary. Circular and elliptical annotations were ignored by the pipeline to ensure that only polygon-based, accurate labels were used for training the segmentation model. This helped the model learn more realistic tumor boundaries and reduced the occurrence of artificial circular effects in its predictions. While this reduced the size of the training set, it improved the quality of supervision, which was more important for reliable model performance.

10. Contributions

This project contributes to the ongoing research on hybrid object detection and segmentation models for brain tumor analysis from MRI data.

A closely related study by Iriawan et al. (2024) introduced a YOLO–U-Net hybrid architecture that combined a CNN-based YOLO model for detection and an FCN-U-Net for segmentation. Their model achieved a correct classification ratio (CCR) of about 97%, showing potential for combining detection and segmentation in a medical context.

However, their paper lacked standard and widely accepted evaluation metrics such as Dice score or Intersection over Union (IoU), which are critical for measuring segmentation accuracy in medical image analysis. Furthermore, the models used in their work were basic: a standard YOLO model (likely YOLOv5 or earlier based on their figures) and a basic U-Net without enhancements.

In contrast, this project provides a much deeper and more detailed implementation and evaluation of a hybrid YOLO–U-Net approach. Specifically:

- Advanced architectures were used: RepVGG-GELAN as the detection model, offering efficient multi-scale detection performance, and a custom EfficientUNet++ segmentation model with nested skip connections and an EfficientNet-B0 encoder for rich spatial feature extraction.
- Thorough evaluation was carried out using key segmentation metrics like Dice score, IoU, and visual inspection, along with examples of model performance in edge cases.
- Dataset cleaning and preprocessing were performed to address annotation noise, particularly the removal of unreliable circular and elliptical tumor annotations.
- Custom tumor cropping, nested tumor handling, and an explicit two-stage pipeline (detection followed by segmentation) were implemented with flexibility for component reuse or standalone evaluation.

Overall, while both works explored the hybrid idea, this project pushes further in terms of architectural depth, implementation clarity, dataset handling, and evaluation quality, making it a more complete and practical contribution to this area of research.

11. Future Work

While the current system achieved strong results in both detection and segmentation, there are still areas that could be improved in future work.

One challenge was detecting very small tumors or those with unclear contrast. The model performed well overall, but certain subtle cases remain difficult to handle consistently. Improving how the system deals with such edge cases could lead to more reliable results, especially in early-stage tumor detection.

Segmentation of tumors with faint or blended boundaries also showed some inconsistency. While the overall masks were accurate, some predictions lacked precision at the edges, particularly when the tumor boundary was not clearly defined in the MRI. Addressing these types of cases could help refine segmentation quality further.

Another area worth exploring is evaluating the system on a wider range of brain tumor datasets. The current model was trained and tested only on the BR35H dataset. Testing on different datasets could help assess how well the model generalizes to new data and clinical conditions.

Lastly, although the two-stage pipeline worked well, its dependence on the detection stage means that segmentation quality is directly affected if a tumor is not detected. Improving the overall robustness of the pipeline would help make the system more reliable in practice.

12. Reflections

This project was a major learning experience and personal challenge. At the start, I had no prior experience in machine learning, deep learning, or even formal research writing. Taking on this project meant learning from the ground up; understanding core machine learning concepts,

navigating frameworks like PyTorch, reading academic papers, and figuring out how to implement models with limited documentation or guidance.

The learning curve was steep, especially when debugging model training, designing experiments, and interpreting results. However, every challenge became a stepping stone. I learned how to evaluate models beyond just metrics, how to clean and preprocess medical data, and how to think critically about results and limitations.

Moreover, writing the report taught me how to structure technical work into clear, academic documentation, a skill I had little exposure to before. Looking back, the technical depth, code organization, and reporting quality I achieved are things I would not have thought possible when I began.

This project pushed me far beyond my comfort zone and showed me the value of persistence, curiosity, and building systems step by step. While the work can still be improved and extended, I am proud of what I managed to accomplish and how far I've come during the process, especially considering the learning challenges and unexpected events in my personal life.

13. Project Management Gannt Chart

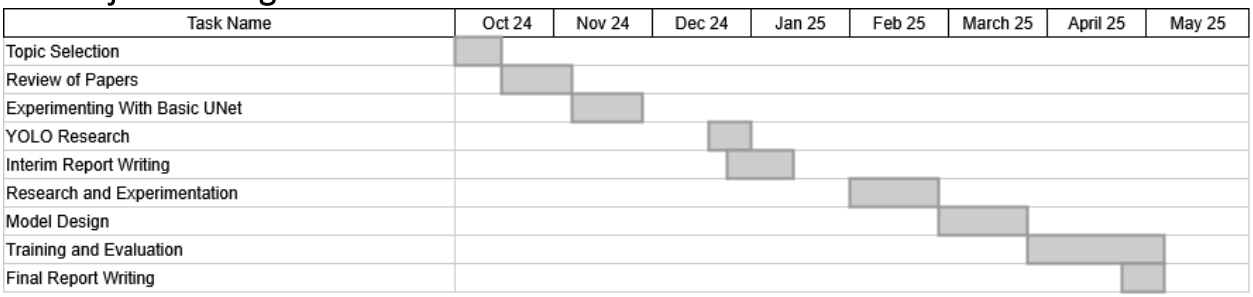

Figure 26: Project Management Gannt Chart

14. Conclusion

This project successfully developed and evaluated a two-stage deep learning pipeline for brain tumor detection and segmentation using MRI scans. By integrating RepVGG-GELAN as a YOLO-based detector and EfficientUNet++ for semantic segmentation, the system effectively combined fast and accurate tumor localization with fine-grained boundary delineation. The modular pipeline design allowed for focused optimization and facilitated interpretability and debugging of each stage.

The models were trained on the BR35H dataset, and despite certain annotation-related challenges, the system achieved high performance across multiple evaluation metrics, including a Dice score of 0.9375 and strong detection precision and recall. Notably, deliberate data cleaning, such as the removal of non-polygon annotations, improved segmentation quality by preventing the model from learning artificial mask shapes.

Qualitative and quantitative evaluations confirmed the strengths of the pipeline, though some limitations remain, particularly in handling faint or small tumors with low boundary

contrast. These edge cases highlight areas for future improvement, including broader dataset evaluation and enhanced handling of low-contrast features.

Overall, the project demonstrates that a hybrid detection-segmentation approach is both practical and effective for medical image analysis. It serves as a foundation for further research and clinical development, showing how lightweight, modular, and high-performance architectures can be adapted to sensitive diagnostic tasks such as brain tumor localization and segmentation.

15. References

- Balakrishnan, T., & Sengar, S. S. (2024). *REPVGG-GELAN: Enhanced GELAN with VGG-STYLE ConVNETs for brain tumour detection* (arXiv:2405.03541v1).
- Hamada, A. (2021). BR35H :: Brain Tumor Detection 2020 [Dataset]. In *Kaggle*. https://www.kaggle.com/datasets/ahmedhamada0/brain-tumor-detection
- Iriawan, N., Pravitasari, A. A., Nuraini, U. S., Nirmalasari, N. I., Azmi, T., Nasrudin, M., Fandisyah, A. F., Fithriasari, K., Purnami, S. W., Irhamah, N., & Ferriastuti, W. (2024). YOLO-UNet architecture for detecting and segmenting the localized MRI brain tumor image. *Applied Computational Intelligence and Soft Computing*, *2024*(1). https://doi.org/10.1155/2024/3819801
- Liu, K., Liu, Y., Su, B., & Tang, H. (2024). Multiscale image denoising algorithm based on UNet3+. *Multimedia Systems*, *30*(2). https://doi.org/10.1007/s00530-024-01284-1
- Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. *Computer Vision and Pattern Recognition*, 779–788. https://doi.org/10.1109/cvpr.2016.91
- Ronneberger, O., Fischer, P., & Brox, T. (2015). U-NET: Convolutional Networks for Biomedical Image Segmentation. In *Lecture notes in computer science* (pp. 234–241). https://doi.org/10.1007/978-3-319-24574-4_28
- Tan, M., & Le, Q., V. (2019, May 28). *EfficientNet: Rethinking model scaling for convolutional neural networks*. arXiv.org. https://arxiv.org/abs/1905.11946
- Zhou, Z., Siddiquee, M. M. R., Tajbakhsh, N., & Liang, J. (2018). UNET++: a nested U-Net architecture for medical image segmentation. *Lecture Notes in Computer Science*, 3–11. https://doi.org/10.1007/978-3-030-00889-5_1